

HAPAG Implementation Plans

HAPAG Form 5A Comparator — Implementation Plans

Document: Per-recommendation implementation plans (R1–R20)
Based on: Comprehensive Code Review (May 2026)
Audience: Lead developer, QA, product owner

Each plan includes: objective, scope, prerequisites, step-by-step tasks, files to change, testing, rollout, and risks.

Dependency Overview

R1 (delete fix) —
R2 (updater) — independent P0 – ship first
R3 (RBAC) —
R4 (hash codes) — may need DB migration + web app coordination
R5 (debug log) — independent, quick

R6 (logging) — should precede or run parallel with R7
R7 (refactor) — enables R12 tests
R8 (shared SQL) — supports R9
R9 (query opt) — independent perf win

R12 (pytest) + R13 (CI) — gate future changes
R14 (pyright) — pairs with R13

Rec	Title	Priority	Effort	Depends on
R1	Fix delete return handling	P0	0.5 day	—
R2	Harden auto-update	P0	2–3 days	—
R3	Role-based permissions	P0	3–5 days	Product: role matrix
R4	Hash access codes	P0	2–4 days	DBA / web app sync
R5	Disable auth debug log	P0	0.5 day	—
R6	Structured logging	P1	2–3 days	—
R7	Split MainWindow	P1	5–10 days	R6 helpful
R8	Extract shared queries	P1	1–2 days	—
R9	Optimize fetch_beneficiaries	P1	2–4 days	R8 helpful
R10	Trim requirements.txt	P1	0.5 day	R12 smoke import test
R11	Unique beneficiary IDs	P1	1 day	—
R12	pytest suite	P2	5–8 days	R6, R8
R13	CI test/lint gates	P2	1 day	R12 partial
R14	pyrightconfig	P2	0.5 day	R13
R15	Code-sign installers	P2	2–5 days	CI secrets, certs
R16	Local/cloud documentation	P2	1 day	—
R17	SQLite foreign keys	P2	1–2 days	sync_engine review
R18	Thin API layer	P3	15+ days	Infra, web team

Rec	Title	Priority	Effort	Depends on
R19	Audit trail	P3	3–5 days	R3, optional R18
R20	Encrypted runtime config	P3	5–8 days	R15 trust model

R1 — Fix Delete Return-Value Handling

Objective

Ensure the UI only reports success when `delete_beneficiary_cascade()` actually returns `True`.

Scope

- **In:** `main_window.py` missing-Excel delete path
- **Out:** `db_duplicate_dialog.py` already checks `ok, err` — use as reference pattern

Prerequisites

None.

Implementation Steps

1. **Audit all delete call sites** (grep `delete_beneficiary_cascade`):
 - `backend/ui/main_window.py` (~1289) — **broken**
 - `backend/ui/components/db_duplicate_dialog.py` (~53) — **correct pattern**
2. **Fix `main_window.py_on_missing_excel_action`:**

```
ok, err = delete_beneficiary_cascade(record.get('beneficiary_id'))
if not ok:
    QMessageBox.critical(self, "Database Error", f"Failed to delete record:\n{err}")
    return
record['_deleted_from_db'] = True
if hasattr(widget, 'mark_as_resolved'):
    widget.mark_as_resolved()
QMessageBox.information(self, "Deleted", f"Successfully deleted {name}.")
```

3. **Optional:** Add `if not beneficiary_id`: guard before calling DB.

4. **Standardize helper** in `core/database.py` (optional):

```
def delete_beneficiary_or_raise(beneficiary_id: str) -> None:
    ok, err = delete_beneficiary_cascade(beneficiary_id)
    if not ok:
        raise RuntimeError(err or "Delete failed")
```

Files to Change

File	Change
<code>backend/ui/main_window.py</code>	Check <code>(ok, err)</code> tuple
<code>backend/ui/components/db_duplicate_dialog.py</code>	Verify parity (no change if OK)

Testing

Test	Steps
Manual — success	Delete a test beneficiary with no children; confirm DB row gone and success dialog
Manual — failure	Use invalid beneficiary_id; confirm error dialog, row not marked resolved
Unit (R12)	Mock delete_beneficiary_cascade returning (False, "FK constraint"); assert no success message

Acceptance Criteria

- ☐ Success dialog only when ok is True
- ☐ Error dialog shows err when ok is False
- ☐ Grid row not marked resolved on failure

Risks

Risk	Mitigation
Exception path still shows generic error	Keep try/except only around unexpected failures, not for expected (False, err)

Effort: 2–4 hours

R2 — Harden Auto-Update (Zip Validation + Checksum)

Objective

Prevent zip-slip attacks and detect tampered release artifacts before overwriting the installed application.

Scope

- **In:** updater.py, main_window.py update install flow, CI release workflow
- **Out:** Full code-signing (see R15)

Prerequisites

- GitHub release process owned by team
- Agreement on checksum publication format (release notes vs .sha256 file per asset)

Implementation Steps

1. Add safe extraction in backend/core/updater.py:

```
def safe_extract_zip(zip_path: str, dest_dir: str) -> str:
    dest_real = os.path.realpath(dest_dir)
    with zipfile.ZipFile(zip_path, 'r') as zf:
        for member in zf.namelist():
            target = os.path.realpath(os.path.join(dest_dir, member))
            if not target.startswith(dest_real + os.sep) and target != dest_real:
                raise ValueError(f"Unsafe path in archive: {member}")
```

```

        zf.extractall(dest_dir)
# ... existing nested-folder detection ...

```

2. **Download checksum file** in `DownloadUpdateThread.run()`:

- After resolving `download_url`, fetch `{asset_name}.sha256` or read checksums from release body
- Compute SHA-256 of downloaded zip; compare before extract
- On mismatch: delete zip, emit error signal

3. **CI: publish checksums** in `.github/workflows/build.yml` release job:

```

- name: Generate SHA256 checksums
  run: |
    sha256sum hapag_comparator_* > checksums.txt

```

Attach `checksums.txt` to GitHub release.

4. **Abort install on validation failure** in `main_window.py` — do not spawn `.bat/.sh` if download thread errors.

5. **Document** operator steps in `backend/ui/manual.html` — “Updates verified via SHA256”.

Files to Change

File	Change
<code>backend/core/updater.py</code>	<code>safe_extract_zip</code> , checksum verify
<code>backend/ui/main_window.py</code>	Block install on failed validation
<code>.github/workflows/build.yml</code>	Emit <code>checksums.txt</code> on release

Testing

Test	Steps
Unit	Zip with <code>../evil.txt</code> member raises <code>ValueError</code>
Unit	Wrong checksum rejects extract
Manual	Full update on Windows/Linux with valid release
Security	Craft malicious zip in temp; confirm rejection

Acceptance Criteria

- ☐ No `extractall` without per-member path check
- ☐ Checksum verified before extraction
- ☐ CI publishes checksums for all platform artifacts
- ☐ Failed validation shows user-visible error; install dir unchanged

Risks

Risk	Mitigation
Checksum file missing on old releases	Fall back: block update or require minimum version
HTTPS MITM	Checksum reduces risk; R15 adds signing

Effort: 2–3 days

R3 — Enforce Role-Based Permissions

Objective

Align with BRD §7.3.2: only authorized roles can perform destructive or high-impact operations.

Scope

Gate these actions (minimum):

Action	Location
Hard delete beneficiary	main_window.py, db_duplicate_dialog.py
Delete Excel rows	excel_duplicate_dialog.py
Bulk sync baseline	main_window.py (on_bulk_sync_*)
Add beneficiary	main_window.py _on_missing_db_action
Bulk site transfer	bulk_transfer_window.py
Full replication	settings_dialog.py

Prerequisites

- **Product decision:** Role → permission matrix (example below)
- Confirm users.role values in production DB (e.g. admin, encoder, supervisor)

Proposed Permission Matrix (draft — confirm with stakeholders)

Permission	admin	supervisor	encoder
scan	✓	✓	✓
sync_baseline	✓	✓	✓
bulk_sync	✓	✓	✗
add_beneficiary	✓	✓	✓
edit_discrepancy	✓	✓	✓
delete_beneficiary	✓	✗	✗
delete_excel_row	✓	✓	✗
bulk_transfer	✓	✓	✗
full_replication	✓	✗	✗
settings_mode_switch	✓	✗	✗

Implementation Steps

1. Create backend/core/permissions.py:

```
ROLE_PERMISSIONS: dict[str, set[str]] = { ... }

def has_permission(role: str | None, permission: str) -> bool:
    if not role:
        return False
    return permission in ROLE_PERMISSIONS.get(role.lower(), set())
```

2. Create backend/ui/auth_guard.py:

```
def require_permission(parent, user: dict, permission: str) -> bool:
    if has_permission(user.get("role"), permission):
        return True
    QMessageBox.warning(parent, "Access Denied", ...)
    return False
```

3. Apply at action entry points — first line of handlers:

```
if not require_permission(self, self.current_user, "delete_beneficiary"):
    return
```

4. Disable buttons at scan complete when permission missing (better UX than dialog-only).

5. Pass current_user into child windows (BulkTransferWindow, dialogs) if not already.

6. Server-side (future): If web app shares DB, add DB triggers or API checks — document as gap until R18.

Files to Change

File	Change
backend/core/permissions.py	New — role map
backend/ui/auth_guard.py	New — UI helper
backend/ui/main_window.py	Guards on bulk sync, delete, add
backend/ui/bulk_transfer_window.py	Guard + disable UI
backend/ui/components/*_dialog.py	Guards
backend/ui/components/settings_dialog.py	Replication guard

Testing

Test	Steps
Unit	has_permission("encoder", "delete_beneficiary") → False
Manual	Login as each role; verify buttons hidden/disabled
Manual	Attempt delete as encoder — blocked

Acceptance Criteria

- ☐ Documented role matrix approved by product owner
- ☐ All destructive actions gated
- ☐ UI reflects permissions before and after scan
- ☐ Access denied message is clear

Risks

Risk	Mitigation
Role strings inconsistent in DB	Normalize with .lower().strip(); log unknown roles
Local SQLite mode bypasses server policy	Same client checks; replication still pulls users.role

Effort: 3–5 days (includes stakeholder sign-off)

R4 — Hash Access Codes at Rest

Objective

Store access codes as one-way hashes; never compare plaintext in SQL.

Scope

- Desktop login (`authenticate_user`)
- **Requires coordination:** Web app user management / password reset flows

Prerequisites

- Choose algorithm: **bcrypt** (add `bcrypt` to requirements) or `argon2-cffi`
- DBA runs migration on `users.access_code` column (widen to `VARCHAR(255)`)
- Migration script for existing plaintext codes

Implementation Steps

1. Add `backend/core/password_utils.py`:

```
import bcrypt

def hash_access_code(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def verify_access_code(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())
```

2. Change `authenticate_user` in `database.py`:

- Fetch user by email only (when Google flow) or by email list
- Verify code in Python with `verify_access_code`
- **Do not** `WHERE access_code = %s` with plaintext

3. Migration script `backend/scripts/migrate_access_codes.py`:

- `SELECT user_id, access_code FROM users WHERE access_code NOT LIKE '$2%'`
- Hash each; `UPDATE users SET access_code = %s`
- Run once on staging, then production

4. **Web app alignment:** Document that new users get hashed codes; admin tools must use same hasher.

5. **Optional:** Rate limiting via DB `login_attempts` table (server-side) — out of scope for desktop-only fix.

Files to Change

File	Change
<code>backend/core/password_utils.py</code>	New
<code>backend/core/database.py</code>	<code>authenticate_user</code> rewrite
<code>backend/requirements.txt</code>	Add <code>bcrypt</code>

File	Change
backend/scripts/migrate_access_codes.py	New
Web app (external)	Same hashing on user create/update

Testing

Test	Steps
Unit	Hash + verify round-trip
Unit	Wrong code fails
Migration	Staging: migrate 3 users; login still works
Regression	Google OAuth + access code flow

Acceptance Criteria

- ☐ No plaintext access codes in DB after migration
- ☐ Login works for migrated users
- ☐ New users stored hashed from web app + desktop admin tools

Risks

Risk	Mitigation
Web app still writes plaintext	Coordinate release; feature flag
Migration partial failure	Transaction per batch; backup first

Effort: 2–4 days (+ web team)

R5 — Disable Auth Debug Log in Production

Objective

Stop writing OAuth debug traces to `~/hapag_auth_debug.log` in release builds.

Implementation Steps

1. Add build flag in `core/version.py` or inject at CI:

```
IS_PRODUCTION_BUILD = getattr(sys, 'frozen', False)
```

2. Replace `log_debug` in `login_window.py`:

```
def log_debug(self, msg):
    if not IS_PRODUCTION_BUILD:
        # existing file write
```

3. Or use **R6 logging** at `DEBUG` level only when env `HAPAG_DEBUG=1`.
4. Delete existing log files in UAT checklist (one-time user comms).

Files to Change

File	Change
backend/ui/login_window.py	Guard log_debug
backend/core/version.py	Optional DEBUG_BUILD flag

Testing

- Frozen build: no file created after login
- Dev mode: log still available when debugging

Effort: 2–4 hours

R6 — Structured Logging (Replace print + bare except)

Objective

Centralized logging with levels; replace bare except : with specific exceptions.

Implementation Steps

1. Create `backend/core/logging_config.py`:

```
import logging
from logging.handlers import RotatingFileHandler

def setup_logging():
    log_dir = ... # same AppData as app_settings
    logging.basicConfig(...)
    # RotatingFileHandler 2MB x 5
```

2. Call `setup_logging()` in `app.py` before UI.

3. Replace `print(f"[TAG]")` in `database.py`, `sync_engine.py`, `parser.py` with:

```
logger = logging.getLogger(__name__)
logger.error("...", exc_info=True)
```

4. **Bare except remediation** (28 sites) — priority order:
 - `parser.py` (8) — use except (ValueError, TypeError): for dates
 - `login_window.py` (7) — log and continue or re-raise
 - `database.py`, `workers.py`, UI components — same pattern
5. **Wire UI console** in `main_window.py` to `logging.Handler` emitting to `QTextEdit`.

Files to Change

File	Change
backend/core/logging_config.py	New
backend/app.py	Init logging
All modules with <code>print(/ except:</code> Incremental PRs	

Testing

- Trigger DB error; verify log file contains stack trace
- UI console shows WARNING+

Acceptance Criteria

- ☐ Zero bare except: in backend/ (enforced by ruff rule E722)
- ☐ Log file under AppData with rotation

Effort: 2–3 days

R7 — Split MainWindow into Controllers

Objective

Reduce main_window.py (~1,296 lines) into testable, focused modules.

Target Structure

```
backend/ui/
  main_window.py          # Layout, tabs, wiring only (~300 lines)
  controllers/
    scan_controller.py    # File selection, ScanWorker, results routing
    sync_controller.py    # bulk sync, single sync, baseline updates
    update_controller.py  # auto-update threads + install scripts
    duplicate_controller.py
    resolution_controller.py # name/bday resolve dialogs
```

Implementation Steps

1. **Extract UpdateController first** (smallest coupling) — move lines ~60–155 update logic.
2. **Extract ScanController** — start_scan, worker signals, tab population.
3. **Extract SyncController** — all on_bulk_sync_*, _do_bulk_sync, single-record sync.
4. **Keep MainWindow as facade** — owns widgets; controllers receive (main_window) or callbacks.
5. **Use signals/slots** between controllers and main window for decoupling.
6. **No behavior change** in this refactor — pure move; R12 tests lock behavior.

Files to Change

File	Change
backend/ui/main_window.py	Shrink
backend/ui/controllers/*.py	New (4–5 files)

Testing

- Full manual UAT script: scan → bulk sync → delete → update check
- After R12: integration tests on controller public methods

Risks

Risk	Mitigation
Regression in signal wiring	One controller per PR; don't big-bang
Effort: 5–10 days (phased PRs)	

R8 — Extract Shared SQL Queries

Objective

Single source of truth for sites list and beneficiary fetch queries.

Implementation Steps

1. Add to `database.py` (or `backend/core/queries.py`):

```
SITES_QUERY = """SELECT s.site_id, ... ORDER BY s.site_name"""

def get_sites():
    return _execute_query(SITES_QUERY)

def fetch_beneficiaries(site_id: str | None = None) -> list:
    sql = BENEFICIARIES_BASE_QUERY
    args = []
    if site_id:
        sql += " WHERE b.site_id = %s"
        args.append(site_id)
    ...
```

2. Remove duplicates from `parser.py` (~223) and `workers.py` (~51) — import `get_sites`.
3. Add `fetch_beneficiaries_for_sites(site_ids: list)` if multi-site scan filter needed later.

Files to Change

File	Change
<code>backend/core/database.py</code> or <code>queries.py</code>	Central SQL
<code>backend/core/parser.py</code>	Remove duplicate
<code>backend/ui/workers.py</code>	Remove duplicate

Effort: 1–2 days

R9 — Optimize `fetch_beneficiaries()`

Objective

Cut scan-time DB load; avoid 12 correlated subqueries per row.

Implementation Steps

1. **Rewrite query** using join + subquery for latest baseline:

```
SELECT b.*, s.site_name, ..., bl.weight, bl.height, ...
FROM beneficiaries b
LEFT JOIN sites s ON ...
LEFT JOIN (
  SELECT beneficiary_id, MAX(created_at) AS max_created
  FROM baseline_info GROUP BY beneficiary_id
) latest ON ...
LEFT JOIN baseline_info bl ON bl.beneficiary_id = latest.beneficiary_id
AND bl.created_at = latest.max_created
```

2. **Add optional site_id filter** (R8) — when Excel files map to one site, pass filter from ScanWorker.
3. **Benchmark** on staging: record row count, query time before/after.
4. **Verify SQLite wrapper** still works with same SQL (test local mode).
5. **Consider materialized view** on server (DBA) — long-term if still slow.

Files to Change

File	Change
backend/core/database.py	New query
backend/ui/workers.py	Pass site filter when known

Testing

- Compare scan results row counts match old query (snapshot test)
- Time scan with 10k beneficiaries — target <50% duration reduction

Effort: 2–4 days

R10 — Trim requirements.txt

Objective

Remove unused packages; reduce bundle size and attack surface.

Implementation Steps

1. **Confirm unused** (already verified): fastapi, uvicorn, starlette, python-multipart.
2. **Evaluate fuzzy stack:** Grep fuzzywuzzy / fuzz — migrate to RapidFuzz only; remove fuzzywuzzy, python-Levenshtein if unused.
3. **Regenerate lockfile:**

```
pip install -r requirements.txt
pyinstaller hapag_comparator.spec # smoke build
```

4. Run import smoke test:

```
# backend/tests/test_imports.py
def test_core_imports():
    import app # noqa
```

Files to Change

File	Change
backend/requirements.txt	Remove unused
backend/core/parser.py	RapidFuzz-only if needed

Effort: 0.5 day

R11 — Beneficiary ID Uniqueness Check

Objective

Prevent duplicate beneficiary_id on insert.

Implementation Steps

1. Replace generate_beneficiary_id() with:

```
def generate_unique_beneficiary_id(cursor, max_attempts=10) -> str:
    for _ in range(max_attempts):
        bid = _random_id()
        cursor.execute("SELECT 1 FROM beneficiaries WHERE beneficiary_id = %s LIMIT
1", (bid,))
        if not cursor.fetchone():
            return bid
    raise RuntimeError("Could not generate unique beneficiary_id")
```

2. Use in add_beneficiary_to_db and baseline info_id generation.

3. **Optional:** DB UNIQUE constraint on beneficiaries.beneficiary_id (verify schema).

4. **Increase length** to 12 chars if collision anxiety at scale (product decision).

Files to Change

File	Change
backend/core/database.py	generate_unique_beneficiary_id

Effort: 1 day

R12 — Introduce pytest Suite

Objective

Automated regression tests for core logic without live DB.

Implementation Steps

1. Add structure:

```
backend/tests/  
  conftest.py          # fixtures, mock DB connection  
  test_parser.py  
  test_permissions.py  
  test_password_utils.py  
  test_updater_safe_extract.py  
  test_anthro_utils.py  
  test_database_delete.py # mock cursor
```

2. Add dev deps requirements-dev.txt: pytest, pytest-qt (optional), pytest-mock.

3. Priority tests:

- split_beneficiary_name / calculate_match_score edge cases
- safe_extract_zip malicious paths
- has_permission matrix
- delete_beneficiary_cascade return handling (mock)

4. Do not hit production DB in CI — all mocked.

Files to Change

File	Change
backend/tests/**	New
requirements-dev.txt	New

Effort: 5–8 days (incremental)

R13 — Extend CI (Test / Lint / Typecheck)

Objective

Gate merges/releases on automated quality checks.

Implementation Steps

1. Add .github/workflows/quality.yml:

```
on: [push, pull_request]  
jobs:  
  quality:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-python@v5  
        with: { python-version: '3.12' }  
      - run: pip install -r backend/requirements.txt -r requirements-dev.txt ruff
```

pyright

- run: `pytest backend/tests -q`
- run: `ruff check backend`
- run: `pyright backend`

2. **Optional:** Require quality job before release workflow (branch protection).

3. Add `ruff.toml` with `select = ["E", "F", "E722"]`.

Files to Change

File	Change
<code>.github/workflows/quality.yml</code>	New
<code>ruff.toml</code>	New

Effort: 1 day

R14 — Strengthen `pyrightconfig.json`

Implementation Steps

```
{
  "include": ["backend"],
  "exclude": ["backend/dist", "backend/build", "backend/_internal"],
  "extraPaths": ["/backend"],
  "typeCheckingMode": "basic",
  "reportMissingImports": true
}
```

Fix top N errors incrementally; don't block CI on strict initially.

Effort: 0.5 day setup + ongoing

R15 — Code-Sign Windows/macOS Installers

Objective

Reduce SmartScreen/Gatekeeper warnings; strengthen update trust chain.

Implementation Steps

1. **Windows:** Obtain Authenticode cert; sign `hapag_comparator.exe` and Inno Setup output in CI:

- uses: `sslcom/Actions@v2` # or `signtool` with secret cert

2. **macOS:** Apple Developer ID; codesign + notarize DMG (requires macOS runner secrets).

3. **Store secrets** in GitHub Actions: `CSC_LINK`, `CSC_KEY_PASSWORD`, Apple API key.

4. **Document** internal cert renewal calendar.

Prerequisites

- Budget for certificates
- Legal entity for Apple/Microsoft developer accounts

Effort: 2–5 days (+ procurement)

R16 — Document Local vs Cloud Mode

Objective

Reduce operator error for replication, sync, and conflict behavior.

Implementation Steps

1. **Create docs/OPERATOR_GUIDE.md:**
 - When to use cloud vs local
 - First-time setup: full replication
 - Auto-sync interval meaning
 - What happens if cloud unavailable
 - Conflict policy (last-write-wins from sync_engine)
2. **Embed summary in backend/ui/manual.html** (in-app).
3. **Add tooltips** in settings_dialog.py next to mode toggle.

Effort: 1 day

R17 — Enable SQLite Foreign Keys (Local Mode)

Objective

Improve referential integrity in offline DB.

Implementation Steps

1. **Review sync_engine.py** — why `PRAGMA foreign_keys=OFF` was set; fix ordering if that was the reason.
2. **Change database.py _get_local_connection:**

```
conn.execute("PRAGMA foreign_keys=ON")
```
3. **Run full replication test** — import 27 tables; verify no FK violations.
4. **Add migration** for existing local DBs: `VACUUM` / re-replicate option in settings.

Risks

Risk	Mitigation
Sync inserts out of order	Fix sync_engine table order; keep OFF only during bulk import transaction
Effort: 1–2 days	

R18 — Thin API Layer (Long-Term)

Objective

Move auth, destructive ops, and audit to a server API instead of direct MySQL from desktop.

Phased Approach

Phase	Deliverable
1	OpenAPI spec for login, beneficiary CRUD, sync
2	FastAPI service (reuse trimmed deps) behind HTTPS + JWT
3	Desktop database.py → HTTP client adapter
4	Deprecate direct MySQL credentials in binary

Prerequisites

- Hosting, TLS, API auth design
- Web team alignment

Effort: 15+ days — separate project charter

R19 — Audit Trail Table

Objective

Accountability for sync, delete, add, and role-gated actions.

Implementation Steps

1. DB migration:

```
CREATE TABLE desktop_audit_log (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id VARCHAR(50),
  action VARCHAR(50),
  entity_type VARCHAR(50),
  entity_id VARCHAR(50),
  details JSON,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

2. Add backend/core/audit.py:

```
def log_action(user_id, action, entity_type, entity_id, details=None):  
    ...
```

3. **Call from:** delete, add_beneficiary, sync_baseline, bulk_transfer (after R3).

4. **Admin viewer** — optional future web report; out of scope for v1.

Dependencies

- R3 (know user_id on each action)
- Optional R18 for centralized API logging

Effort: 3–5 days

R20 — Encrypted Runtime Config (Not Compiled Secrets)

Objective

Avoid baking CLOUD_DB_PASSWORD into PyInstaller binary where possible.

Implementation Steps

1. **First-login cloud setup wizard** — prompt admin for DB credentials once; store with OS keyring:
 - Windows: keyring + Credential Manager
 - macOS: Keychain
 - Linux: Secret Service
2. **CI builds** — use ephemeral injection only for QA builds; production users enter creds OR use OAuth-only API (R18).
3. **Remove _config.py from binary** — load at runtime from keyring.
4. **Migration:** Existing installs keep working until re-setup.

Files to Change

File	Change
backend/core/secrets_store.py	New — keyring wrapper
backend/core/_config.py	Fallback dev-only
backend/scripts/inject_config.py	QA builds only
backend/ui/setup_wizard.py	New

Effort: 5–8 days — pairs with R18 long-term

Suggested Delivery Sprints

Sprint 1 (Week 1) — P0 Security & Correctness

Day	Items
1	R1, R5
2–4	R2
3–5	R3 (parallel, needs matrix day 1)

Sprint 2 (Week 2) — P0/P1 Foundation

Day	Items
1–3	R4 (with DBA)
2–4	R6
4–5	R8, R10, R11

Sprint 3 (Weeks 3–4) — Quality & Performance

Items
R9, R7 (phase 1–2), R12 (core tests), R13, R14

Sprint 4+ — Ops & Architecture

Items
R15, R16, R17, then R19, R18/R20 as program

Appendix — Quick Reference: Files by Recommendation

Rec	Primary files
R1	ui/main_window.py
R2	core/updater.py, ui/main_window.py, .github/workflows/build.yml
R3	core/permissions.py, ui/auth_guard.py, ui/main_window.py, dialogs
R4	core/database.py, core/password_utils.py, scripts/migrate_access_codes.py
R5	ui/login_window.py
R6	core/logging_config.py, app.py, all backend/**/*.*.py
R7	ui/main_window.py, ui/controllers/*
R8	core/database.py, parser.py, workers.py
R9	core/database.py, workers.py
R10	requirements.txt
R11	core/database.py
R12	backend/tests/
R13	.github/workflows/quality.yml
R14	pyrightconfig.json
R15	.github/workflows/build.yml, scripts/windows_setup.iss
R16	docs/OPERATOR_GUIDE.md, ui/manual.html
R17	core/database.py, core/sync_engine.py
R18	New API service (separate repo or backend/api/)

	Rec	Primary files
R19		core/audit.py, DB migration
R20		core/secrets_store.py, ui/setup_wizard.py

End of implementation plans document.